

SIMATS ENGINEERING

ASSESSMENT TOOL 2 — CASE STUDY ANSWERS

MLA03 — Reinforcement Learning for Environment and Agent

CO1: Analyze RL frameworks and MDP concepts to model decision-making problems

SOLUTIONS

Q1. A logistics company is developing an RL-based warehouse robot that learns to pick, pack, and transport items efficiently while avoiding obstacles and minimizing delays. Analyze how the components of a Markov Decision Process (states, actions, rewards, policy, and environment) can be defined for this system. Design a reward function that balances speed, accuracy, and safety, and evaluate how reward design influences the robot's learning and behavior.

MDP Components

- States: robot position/location on the warehouse grid, item location, battery level, presence of nearby obstacles, current payload status (empty/carrying item).
- Actions: move (up/down/left/right), pick item, place/pack item, wait, recharge.
- Environment: the warehouse floor plan with shelves, obstacles, other moving robots/workers, and dynamic order queue.
- Policy: a mapping from the robot's current state to the best action — e.g., a learned policy that picks the shortest safe path to the next item while avoiding collisions.

Reward Function Design

- +10 for successfully picking/packing an item correctly.
- +5 for reaching the destination faster than the expected time (speed incentive).
- -1 per time step (encourages efficiency, discourages idling).
- -20 for collisions with obstacles/other robots (safety penalty).
- -10 for dropping or mis-packing an item (accuracy penalty).

Analysis — Effect of Reward Design on Behavior

If speed is over-weighted relative to safety, the robot learns aggressive, collision-prone paths. If safety penalties dominate, the robot becomes overly cautious and slow, hurting throughput. A balanced reward (small step penalty + moderate speed bonus + large safety penalty) produces an agent that moves efficiently while still prioritizing collision avoidance — this is the core trade-off the reward function must resolve.

Q2. A streaming platform wants to use Reinforcement Learning to recommend movies dynamically based on user preferences and viewing history. Apply RL concepts to construct a suitable state representation, action space, and reward function. Analyze how different reward signals affect user engagement and evaluate the trade-off between exploration and exploitation.

MDP Components

- State: user's viewing history, genre preferences, recent ratings/clicks, time of day, device type.
- Action space: the set of movies/shows that can be recommended next.
- Reward function: +1 for a click, +3 for completing the video, +5 for a high rating; -1 for skip/ignore within a few seconds.

Analysis — Effect of Reward Signals on Engagement

Rewarding only clicks encourages clickbait-style recommendations that boost short-term engagement but hurt long-term satisfaction (users churn). Rewarding completion/ratings instead encourages recommendations aligned with genuine interest, improving retention. A well-designed system blends short-term (click) and long-term (watch-time, satisfaction) signals.

Exploration vs Exploitation Trade-off

Exploitation means always recommending the genre/titles known to work for a user (safe, but repetitive). Exploration means occasionally recommending something new to discover fresh preferences. Too much exploitation causes a stale 'filter bubble'; too much exploration frustrates users with irrelevant suggestions. An epsilon-greedy or UCB-style strategy — mostly recommending known favorites while occasionally trying novel content — balances discovery with relevance.

Q4. A cybersecurity system uses RL to detect and respond to network attacks in real time. Analyze how states and environment can be modeled for intrusion detection. Design appropriate actions and reward functions, and evaluate the challenges of sparse rewards and real-time decision-making.

Modeling States and Environment

- State: network traffic features — packet rate, source/destination IP patterns, protocol type, connection duration, historical anomaly score.
- Environment: the live network, which is highly dynamic, adversarial (attackers adapt), and non-stationary (traffic patterns evolve over time).

Actions and Reward Function

- Actions: allow traffic, flag as suspicious, block/quarantine connection, alert a human analyst.
- Reward: +10 for correctly blocking a real attack, +1 for correctly allowing legitimate traffic, -10 for a false positive (blocking legitimate traffic), -50 for a false negative (missing a real attack).

Analysis — Challenges

Sparse rewards: true attacks are rare compared to normal traffic, so the agent gets very little positive/negative feedback to learn from, slowing convergence. Real-time decision-making: the agent must act within milliseconds, leaving no time for expensive computation or long deliberation, forcing a trade-off between model complexity and response latency. Both challenges suggest using reward shaping (intermediate rewards for suspicious-pattern detection) and lightweight, fast-inference policies.

Q6. A smart home automation system uses RL to control lighting, heating, and cooling based on occupancy and weather conditions. Design an RL framework including states, actions, and reward function. Analyze how conflicting objectives like comfort and energy saving can be handled and evaluate the impact of reward shaping.

RL Framework

- States: room occupancy, current temperature/humidity, time of day, outdoor weather, appliance status (heating/cooling/lighting on-off).
- Actions: adjust thermostat up/down, turn lights on/off/dim, switch HVAC mode, no change.
- Reward function: +reward for maintaining comfort range (temperature/lighting within user preference) and -penalty proportional to energy consumed.

Analysis — Handling Conflicting Objectives

Comfort and energy-saving directly conflict: maximum comfort (constant heating/cooling/lighting) uses the most energy, while maximum energy-saving sacrifices comfort. This is resolved with a weighted reward: $R = w_1 * (\text{comfort_score}) - w_2 * (\text{energy_used})$, where the weights w_1 , w_2 are tuned to reflect the household's priorities (e.g., higher w_2 for energy-conscious households).

Evaluation — Impact of Reward Shaping

Reward shaping (adding intermediate rewards, e.g., small bonus for pre-cooling before occupants arrive) helps the agent learn faster and produces smoother, more anticipatory behavior instead of reactive on/off switching. Poorly shaped rewards, however, can cause the agent to over-optimize for the shaping signal instead of the true comfort/energy goal.

Q10. A smart grid system uses RL to manage electricity distribution and reduce peak load demand. Analyze how the components of an MDP can be defined in this context. Design a reward function to balance efficiency and reliability, and evaluate the challenges of scalability and real-time learning.

MDP Components

- States: current electricity demand across zones, generation capacity available, time of day, weather forecast, storage/battery levels.
- Actions: increase/decrease supply to a zone, activate backup generation, charge/discharge storage, apply demand-response signals (e.g., dynamic pricing to shift load).
- Environment: the power grid network with fluctuating demand, renewable energy variability (solar/wind), and equipment constraints.

Reward Function Design

- +reward for meeting demand without shortage or blackout (reliability).
- +reward for minimizing energy wasted/over-generation (efficiency).
- -penalty for peak-load spikes and for load shedding/blackouts.

Analysis — Scalability and Real-Time Learning Challenges

Scalability: a real grid has thousands of interconnected nodes, so the state-action space grows extremely large, making tabular RL infeasible — this motivates function approximation (deep RL) or hierarchical/multi-agent RL (one agent per zone/substation). Real-time learning: demand and renewable supply change continuously, so the policy must adapt online without destabilizing the grid; this requires fast inference and safe-exploration techniques (the agent cannot 'experiment' by causing real blackouts) — often addressed via simulation-based pre-training before deployment.